ΔΟΜΕΣ ΔΕΔΟΜΈΝΩΝ
ΕΡΓΑΣΙΑ ΕΞΑΜΗΝΟΥ

• Main:

Αρχικά, ανοίγω το αρχείο commans.txt για το διάβασμα των εντολών και το output.txt για την εκτύπωση των αποτελεσμάτων. Έπειτα, θέτω την μεταβλητή t για την χρονομέτρηση της ώρας εκτέλεσης των εντολών και τις ακέραιες μεταβλητές flMIN,flMAX,flGR,flTREE ,flHASH για να ξέρω αν προηγήθηκε ή όχι η εντολή BUILD για κάθε μία από τις δομές. Στην συνέχεια, δηλώνω και θέτω τις μεταβλητές num,num2,num3 στο -1, οι οποίες θα δεχτούν τους αριθμούς από το αρχείο. Ύστερα, δηλώνω τις συμβολοσειρές c,d και f στις οποίες θα αποθηκεύσω την εντολή, την δομή στην οποία πρέπει να εκτελεστεί και το όνομα του αρχείου από το οποίο θα διαβάσω τα στοιχεία αντίστοιχα. Το πρόγραμμα μετά διαβάζει από το commands.txt δύο λέξεις όπου η πρώτη είναι μία εντολή και η δεύτερη η δομή στην οποία πρέπει να εκτελεστεί. Κάθε φορά που διαβάζει αυτά τα δυο ελέγχει τις εξής περιπτώσεις:

•Αν η εντολή είναι η Build τότε περνάω στην f το όνομα του αρχείου από το οποίο θα διαβάσω τα δεδομένα.

•Αν η εντολή είναι η Search ή η Insert διαβάζω έναν ακέραιο και τον περνάω στον num αλλά στην περίπτωση του γράφου διαβάζω τρείς αριθμούς και τους περνάω στους num,num2,num3.

•Αν η εντολή είναι η Computetheshortestpath διαβάζω δύο ακεραίους και τους περνάω στους num,num2 αντιστοίχως. Αυτή η διαδικασία επαναλαμβάνεται μέχρι να διαβαστούν όλες οι εντολές και δομές από το αρχείο commands.txt. Στο υπόλοιπο μέρος της main.cpp ελέγχω ποια εντολή πρέπει να εκτελεστεί για κάθε εντολή και δομή που διαβάζω. Με εντολές if ,else if βρίσκω σε ποια δομή πρέπει να πάω. Αν η εντολή είναι η Build τότε αλλάζω το αντίστοιχο flag(π.χ. flMIN για την MINHEAP) σε 1 ώστε να ξέρω πως προηγήθηκε η Build. Αν είναι οποιαδήποτε άλλη εντολή εκτός της Build ελέγχω αν έχει προηγηθεί η Build με το flag. Αν έχει

προηγηθεί τότε εκτελείτε κανονικά η εντολή αλλιώς φεύγει από την if και διαβάζει την επόμενη εντολή.

•Minheap-Maxheap: Αρχικά δημιουργώ τις συναρτήσεις parent , leftchild και rightchild οι οποίες μας επιτρέπουν να προελαύνουμε τον σωρό επιστρέφοντας κάθε φορά την θέση του γονιού του αριστερού ή το δεξιού παιδιού αντίστοιχα από το στοιχείο για το οποίο περνάμε την θέση του. Έπειτα δημιουργώ την συνάρτηση overload η οποία περνάει τα στοιχεία του σωρού σε έναν προσωρινό πίνακα με το διπλάσιο μέγεθος του προηγούμενου πίνακα ,διαγράφει τον προηγούμενο και βάζουμε τον πίνακα να δείχνει στον καινούριο πίνακα που φτιάξαμε. Απαραίτητη για την υλοποίηση του Minheap είναι η συνάρτηση Minheapify η οποία δίνει την μορφή σωρού ελαχίστων στο υποδέντρο με ρίζα το στοιχείο που βρίσκεται στην θέση που της περνάμε. Αυτή την λειτουργεία την πραγματοποιεί βρίσκοντας κάθε φορά το ελάχιστο στοιχείο μεταξύ της ρίζας και των δύο παιδιών και αν αυτό δεν είναι η ρίζα το αλλάζει θέση με την ρίζα και επαναλαμβάνει την διαδικασία για το υποδένδρο του μικρότερου στοιχείου μέχρι να βρίσκονται όλα σε σωστή θέση. Για την εισαγωγή στοιχείου ελέγχω αν ο πίνακας τον στοιχείων δεν χωράει άλλα και σε αυτή την περίπτωση καλώ την συνάρτηση overload η οποία θα διπλασιάσει το μέγεθος του πίνακα. Διαφορετικά αν υπάρχει χώρος προσθέτω το στοιχείο στην τελευταία θέση του πίνακα και το αλλάζω θέσεις με τον γονιό του μέχρι να βρεθεί στην σωστή θέση δηλαδή μέχρι ο γονιός του να είναι μικρότερος από αυτόν. Τέλος για την διαγραφή του ελαχίστου στοιχείου το αντικαθιστούμε με το τελευταίο στοιχείου του σωρού και έπειτα πραγματοποιώ τη διαδικασία Heapify για να το επαναφέρω στην μορφή σωρού ελαχίστων. Για τον σωρό μεγίστων η διαδικασία είναι παρόμοια.

•Hashtable: Αρχικά δημιουργώ τη συνάρτηση HandleOverload η οποία φτιάχνει έναν νέο προσωρινό πίνακα με διπλάσιο από το μέγεθος του προηγούμενου και για κάθε θέση του πίνακα παίρνει όλους τους κόμβους της (αν έχει) και τους βάζει στην θέση του νέου πίνακα που αντιστοιχούν βάση της εξίσωσης (τιμη%μεγεθος). Έπειτα υλοποιώ την συνάρτηση Insert η οποία εισάγει το στοιχείο στο Hashtable. Για να πραγματοποιήσει την παραπάνω λειτουργεία βρίσκει την θέση του πίνακα που πρέπει να αποθηκευτεί το στοιχείο βάση της εξίσωσης και αν στην θέση αυτή δεν υπάρχει άλλο στοιχείο απλά βάζει αυτό, ενώ αν υπάρχει ήδη στοιχείο στην θέση τότε προελαύνουμε όλους τους κόμβους που είναι

αλυσοδεμένους σε αυτήν την θέση και όταν βρούμε τον τελευταίο θέτουμε ως επόμενο του το στοιχείο που θέλουμε να εισάγουμε. Για να περάσω τα στοιχεία από το αρχείο στον πίνακα του Hashtable χρησιμοποιώ την Build στην οποία περνώ το όνομα του αρχείου. Η Build διαβάζει τα στοιχεία ένα προς ένα και αν ο αριθμός των στοιχείων είναι μεγαλύτερος από το μισό του μεγέθους του πίνακα καλεί την συνάρτηση HandleOverload όμως αν δεν είναι μεγαλύτερος τότε καλεί την συνάρτηση Insert για να εισαχθεί το στοιχείο στον πίνακα. Τέλος για την αναζήτηση στοιχείου στο Hashtable βρίσκουμε την θέση στην οποία πρέπει να βρίσκεται το στοιχείο αν υπάρχει και αν το στοιχείο δεν βρίσκεται σε αυτήν την θέση προχωράμε στον επόμενο κόμβο που είναι αλυσοδεμένος με αυτήν την θέση μέχρι να βρω το στοιχείο ή μέχρι να μην υπάρχουν άλλοι αλυσοδεμένοι κόμβοι σε αυτήν την θέση. Αν στο τέλος της επανάληψης έχουμε βρει το στοιχείο επιστρέφουμε “SUCCESS” αλλιώς επιστρέφουμε “FAILURE”.

AVLTREE

ΣΗΜΑΝΤΙΚΗ ΣΥΜΕΙΩΣΗ!!!:
Τα αρχεία που χρησιμοποιήθηκαν για να υλοποιηθεί το AVLTREE είναι το Node.h, Node.cpp, tree.h, tree.cpp. Παρακαλώ να αγνοηθούν τυχόν εκτυπώσεις στη κονσόλα καθώς βρίσκονται εκεί αποκλειστικά και μόνο για λόγους επαλήθευσης. Όλες οι λειτουργίες που ζητήθηκαν στην εκφώνηση θα βρίσκονται στο αρχείο εξόδου.

Επεξήγηση Υλοποίησης:
Συνάρτηση Build:
Διαβάζουμε ένα ένα τα στοιχεία από το αρχείο και τα εκχωρούμε σε έναν πίνακα και αν ο αριθμός των κόμβων που δίνονται ξεπερνάει το capacity τότε το διπλασιάζει. Κάθε φορά που πάει να προσθέσει έναν κόμβο πρώτα ελέγχει αν υπάρχει ήδη ο ίδιος κόμβος και μετά το προσθέτει αφού δεν υπάρχει ήδη.

Συνάρτηση Insert: Διαπερνάμε το δέντρο προκειμένου να βρούμε τη σωστή θέση να βάλουμε τον κόμβο αλλάζοντας τη ισορροπία στην επιστροφή της αναδρομής.

Συνάρτηση Calc_h: Υπολογίζουμε το ύψος αναδρομικά κάθε υπόδεντρου του κόμβου και το μεγαλύτερο + 1 γίνεται το ύψος του κόμβου αυτού.

Συνάρτηση diafora: Υπολογίζει την ισορροπία, η οποία δίνεται από το τύπο ύψος_αριστερού_υποδέντρου - ύψος_δεξιού υποδέντρου.

Συνάρτηση leftrotate: Στη συνάρτηση αυτή κάνουμε αριστερή περιστροφή όπου το δεξιό παιδί του γονέα γίνεται η ‘ρίζα’ και ο γονέας γίνεται το αριστερό παιδί του παίρνοντας μαζί ως δεξί παιδί το αριστερό παιδί του παιδιού του.

Συναρτήσεις Setroot,Getroot: Getters και Setters για τη ρίζα.

Συνάρτηση addsize:Αυξάνει το ύψος κατά ένα.

Συνάρτηση rightrotate: Στη συνάρτηση αυτή κάνουμε δεξιά περιστροφή όπου το αριστερό παιδί του γονέα γίνεται η ‘ρίζα’ και ο γονέας γίνεται το δεξί παιδί του παίρνοντας μαζί ως αριστερό παιδί το δεξί παιδί του παιδιού του.

Συνάρτηση lrrotate: Κάνει μια αριστερή περιστροφή στο αριστερό παιδί του γονέα και στη συνέχεια κάνει μια δεξιά περιστροφή στον γονέα.

Συνάρτηση rlrotate: Κάνει μια δεξιά περιστροφή στο δεξί παιδί του γονέα και στη συνέχεια κάνει μια αριστερή περιστροφή στον γονέα.

Συνάρτηση Calc_bf: Υπολογίζει την ισορροπία και στη συνέχεια με βάση αυτή κάνουμε τις περιστροφές που πρέπει ώστε το δυαδικό αυτό δέντρο να γίνει δυαδικό δέντρο AVL.
Συνάρτηση search: Ξεκινάμε από τη ρίζα και με βάση αν είναι μικρότερο ή μεγαλύτερο από κάθε παιδί του εκάστοτε κόμβου προχωράμε στο δέντρο μέχρι να το βρούμε.

Συνάρτηση Findmin:Προχωράμε όσο αριστερά στο δέντρο γίνεται ώστε να βρούμε το ελάχιστο στοιχείο αυτο βασίζεται στο γεγονός ότι αποτελεί δυαδικό δέντρο αναζήτησης.

Συνάρτηση Delete: Αρχικά ψάχνω να βρω το κόμβο αναδρομικά. Μόλις τον βρω στη επιστροφή της αναδρομής εξετάζω τις περιπτώσεις για μονά παιδιά και ανάλογα τα συνδέω με τους παραπάνω κόμβους. Μετά, προκειμένου να διατηρήσουμε την "δυαδικότητα δέντρων αναζήτησης" πάμε να βρούμε τον μικρότερο κόμβο από τα δεξιά του κόμβου που θέλαμε να διαγράψουμε και να τον βάλουμε στη θέση του κόμβου αυτού

για να το κάνουμε τη "ρίζα" του υπόδεντρου με όλα τα άλλα δεξιά υπόδεντρα να είναι μικρότερα από αυτό.

Συνάρτηση maximum:Υπολογίζει ποιος αριθμός ειναι μεγαλύτερος και τον επιστρέφει (χρησιμεύει για τα ύψη).

Συνάρτηση msize:Μειώνει το ύψος κατά ένα.

Χρόνοι και παρατηρήσεις:

- Για είσοδο 1.000 στοιχείων στο AVL από τυχαίους σε σειρά αριθμούς 0-99:

Οι ενέργειες ήταν αυτές:
BUILD AVLTREE a.txt
GETSIZE AVLTREE
SEARCH AVLTREE 100
INSERT AVLTREE 100
DELETE AVLTREE 100
SEARCH AVLTREE 100
FINDMIN AVLTREE
GETSIZE AVLTREE
Ώστε να συμπεράνουμε αν λειτουργούν σωστά οι συναρτήσεις και όντως λειτουργούν. Πράγματι δώσαμε 100 αριθμούς και ήταν όλοι δοαφορετικοί μεταξύ τους αρα ο αριθμός των κόμβων έτυχε να είναι ακριβώς 100και επομένως οι ακμές είναι 99.Ψάξαμε το 100 και δεν το βρήκαμε αφού όντως δεν το προσθέσαμε .Στη συνέχει, το προσθέτουμε και βλέπουμε ότι όντως η εντολή SEARCH το βρίσκει. Έτσι το μόνο που μας μένει είναι δοκιμάσουμε τη DELETE και να δούμε αν το 100 ακόμα υπάρχει και αν

το πλήθος των κόμβων μειώθηκε, τα οποία και τα δύο συνέβησαν. Τέλος, το ελάχιστο στοιχείο είναι πράγματι το 0.

<u>Χρόνοι</u>:

BUILD: 0.005 sec
GETSIZE:1e-05 sec
SEARCH:3.3e-05 sec
INSERT:1.6e-05 sec
DELETE:3e-06 sec
SEARCH:1e-05 sec
FINDMIN: 1e-06 sec
GETSIZE:8e-06

```
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ BUILD: 0.005852
Size of : 100 99
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ GETSIZE: 1e-05
Search happend with FAILURE
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ SEARCH: 3.3e-05
Succesfull insertion of number
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ INSERT: 1.6e-05
Succesfull deletion
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ DELETE: 3e-06
Search happend with FAILURE
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ SEARCH: 1e-05
Minimum value is : 0
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ FINDMIN: 1e-06
Size of : 100 99
```

- Για είσοδο 100.000 στοιχείων στο AVL από τυχαίους σε σειρά αριθμούς 0-99.999:

Απο ότι φαίνεται πάλι η προσθήκη του 100.000 γίνεται σωστά αφού η SEARCH επιστρέφει τα σωστά αποτελέσματα.

<u>Χρόνοι</u>:

BUILD: 2 λεπτά και 2 sec
GETSIZE:0.02 sec
SEARCH:9e-06 sec
INSERT: 0.01 sec
DELETE:0.007 sec
SEARCH:1.1e-05 sec
FINDMIN: 3e-06 sec
GETSIZE:0.02 sec

```
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ BUILD: 122.822
Size of : 63202 63201
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ GETSIZE: 0.024809
Search happend with FAILURE
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ SEARCH: 9e-06
Succesfull insertion of number
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ INSERT: 0.010732
Succesfull deletion
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ DELETE: 0.007
Search happend with FAILURE
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ SEARCH: 1.1e-05
Minimum value is : 0
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ FINDMIN: 3e-06
Size of : 63202 63201
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ GETSIZE: 0.026913
```

- Για είσοδο 1.000.000 στοιχείων στο AVL από τυχαίους σε σειρά αριθμούς 0-999.999:

Απο ότι φαίνεται πάλι η προσθήκη του 1.000.000 γίνεται σωστά αφού η SEARCH επιστρέφει τα σωστά αποτελέσματα.

```
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ BUILD: 448.382
Size of : 99992 99991
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ GETSIZE: 0.04476
Search happend with FAILURE
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ SEARCH: 1.3e-05
Succesfull insertion of number
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ INSERT: 0.019593
Succesfull deletion
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ DELETE: 0.017937
Search happend with FAILURE
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ SEARCH: 1.6e-05
Minimum value is : 0
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ FINDMIN: 4e-06
Size of : 99992 99991
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ GETSIZE: 0.043053
```

Χρόνοι:
BUILD: 7 λεπτά και 46 sec
GETSIZE:0.04 sec
SEARCH:1.3e-05 sec
INSERT: 0.01 sec
DELETE:0.017 sec
SEARCH:1.6e-05 sec
FINDMIN: 4e-06 sec
GETSIZE:0.04 sec

Η Insert δουλεύει σε όλες τις περιπτώσεις ανεξάρτητα που δεν χρησιμοποιούμε search για επιβεβαίωση. Αφού παίρνουμε το μήνυμα επιτυχής προσθήκη και όταν διαγράφουμε τον αριθμό το size είναι ίδιο με το αρχικό σημαίνει ότι το size πριν είχε αυξηθεί επομένως η Insert εκτελέστηκε σωστά.

GRAPH

ΣΗΜΑΝΤΙΚΗ ΣΥΜΕΙΩΣΗ!!!:

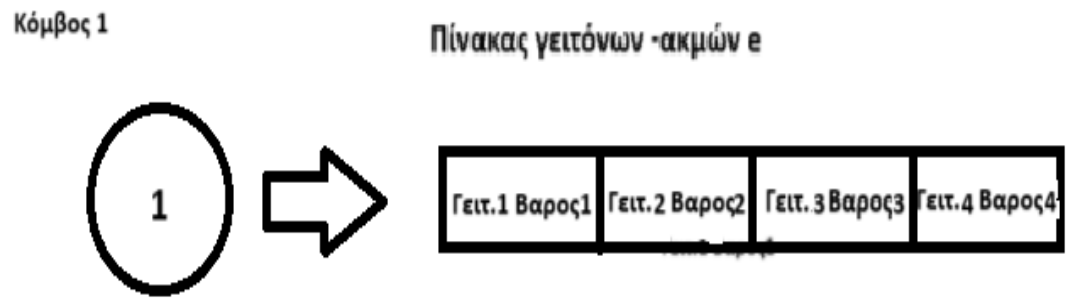
Τα αρχεία που χρησιμοποιήθηκαν για να υλοποιηθεί το AVLTREE είναι το Node.h, Node.cpp, Graph.h, Graph.cpp. Παρακαλώ να αγνοηθούν τυχόν εκτυπώσεις στη κονσόλα καθώς βρίσκονται εκεί αποκλειστικά και μόνο για λόγους επαλήθευσης. Όλες οι λειτουργίες που ζητήθηκαν στην εκφώνηση θα βρίσκονται στο αρχείο εξόδου.

Υλοποίηση Γράφου:

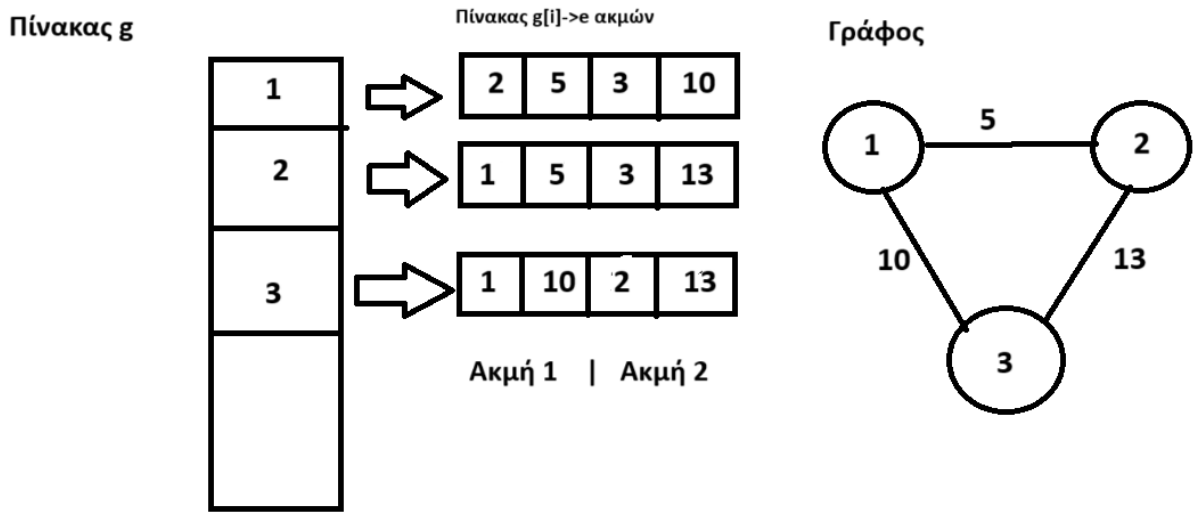
Κάθε γράφος θα αναπαριστάται από ένα δυναμικό πίνακα τύπου Node ** g όπου θα περιέχει τους κόμβους με τη σειρά που δόθηκαν στην είσοδο (δεν περιέχει διπλούς κόμβους). Αυτοί οι κόμβοι είναι επίσης δυναμικοί δηλαδή αποθηκεύονται στο Heap με αποτέλεσμα να μην υπάρχουν αντίγραφα στον γράφο αλλά οι ίδιοι κάθε φορά(δηλαδή στον g αποθηκεύεται η διεύθυνσή τους).
Κάθε κόμβος έχει έναν δικό του πίνακα με ακμές επίσης δυναμικό, που περιέχει σε κάθε κελί την ακμή με την οποία συνδέεται.

Κάθε τέτοια ακμή περιέχει τον γείτονα κόμβο δηλαδή τον κόμβο στον οποίο πηγαίνουμε μέσω της ακμής και το βάρος της.

Ένα παράδειγμα κόμβου είναι το εξής:



Ένα παράδειγμα γράφου είναι το εξής:



Επίσης, Κάθε κόμβος Node έχει getter για το ονομά του Getnumb() Και κάθε ακμή έχει getter για τον κόμβο που πηγαίνουμε κάθε φορά Getv() αλλά και για το βάρος Getw().

<u>Επεξήγηση Υλοποίησης:</u>

<u>Συνάρτηση Build:</u> Σε αυτή τη συνάρτηση διαβάζουμε από το αρχείο τα δεδομένα μας και στη συνέχεια ελέγχουμε αν ο αριθμός των κόμβων που διαβάζουμε υπερβαίνει το αρχικό μας capacity σε κόμβους αλλά και σε γείτονες . Αν συμβεί αυτό τότε διπλασιάζουμε το capacity και ύστερα δημιουργούμε νέους πίνακες με αυτό το capacity. Ύστερα αφού διαβάσουμε τον κόμβο ελέγχουμε αν ήδη υπάρχει αλλιώς τον

δημιουργούμε. Επίσης ελέγχουμε και αν αυτή η ακμή ήδη υπάρχει. Σε περίπτωση που δεν υπάρχει προσθέτουμε τον αντίστοιχο γείτονα και το βάρος της ακμής στον δυναμικό πίνακα γειτώνων και στους δύο κόμβους.
Συνάρτηση Insert: Η υλοποίησή της είναι ακριβώς η ίδια με τη Build

Συνάρτηση shortest_path():Σε αυτή την συνάρτηση εφαρμόζουμε τον αλγόριθμο του Dijkstra. Αρχικοποιούμε τις μεταβλητές του κάθε κόμβου όπως το αν τον επισκεφτήκαμε (visited) και την απόστασή τους που την αρχικοποιούμε άπειρη (dist). Ύστερα, μέχρι όλοι οι κόμβοι να έχουν επισκεφτεί βρίσκουμε το μικρότερο dist και γι αυτόν τον κόμβο ενημερώνουμε το dist όλων των γειτόνων του εφόσον το βάρος της ακμής τους + το dist του κόμβου από τον οποίον ξεκινάμε είναι μικρότερο από το dist του γείτονα εκείνη τη στιγμή . Στο τέλος επιστρέφουμε το dist του κόμβου που θέλαμε να πάμε.

Συνάρτηση GetSize():Επιστρέφουμε τον αριθμό των κόμβων και τον αριθμό των ακμών που ισούται με όλες οι ακμές που έχουμε στους πίνακες /2 γιατί εμφανίζονται δύο φορές η κάθε μία.

Συνάρτηση Delete(): Στην Delete ψάχνουμε το που βρίσκονται οι κόμβοι που συνδέουν την ακμή που θέλουμε να διαγράψουμε στον πίνακα κόμβων μας g αλλά και στους γείτονες κάθε κόμβου. Στη συνέχεια, τους αφαιρούμε κάνοντας shift τα στοιχεία και μειώνοντας τον αριθμό των γειτόνων κάθε κόμβου. Στο τέλος ελέγχουμε αν αυτοί οι κόμβοι δεν έχουν κανένα γείτονα ,διότι αν ισχύει αυτό θα πρέπει να τους αφαιρέσουμε και από τον g αφού δεν ενώνονται με κανέναν κόμβο πια.

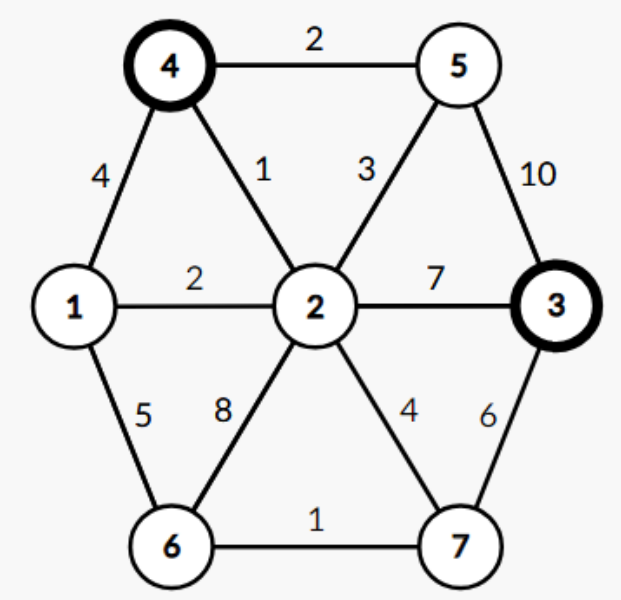
Συνάρτηση DFS(),CCC(): Κάνουμε depth first search όπου για κάθε κόμβου που υπάρχει τον επισκεπτόμαστε και πηγαίνουμε σε όλους τους πιθανούς του γείτονες . Έτσι καταλήγουμε με τον αριθμό των συνιστωσών που έχει ένας γράφος , καθώς μέσω ενός κόμβου κάθε φορά διασχίζουμε όλα τα πιθανά μονοπάτια. Και αν μετά κάποιος κόμβος δεν έχει επισκεφτεί τότε σημαίνει ότι έχουμε μια παραπάνω συνιστώσα.

Συνάρτηση min_span_tree(): Σε αυτήν την συνάρτηση εφαρμόζουμε τον αλγόριθμο του Kruskal όπου ταξινομούμε τις ακμές σε έναν ενιαίο πίνακα και στη συνέχεια χρωματίζουμε κάθε ακμή με διαφορετικό χρώμα. Αν δύο ακμές τύχουν να βρίσκονται σε ίδια ακμή και είναι σε διαφορετικά σύνολα (τα οποία χωρίζονται με ένα κριτήριο τον γονέα- αρχικό κόμβο) τότε ενώνονται αυξάνοντας έτσι και το συνολικό βάρος του MST .

Σε αυτό το έργο βοηθάνε οι συναρτήσεις find parent και union parent που βοηθάνε να ξεχωρίσουμε με κάποιο κριτήριο τις ακμές αλλά και να αποφύγουμε πιθανούς κύκλους. Η check βοηθάει στο να μην εισαχθεί μια ακμή δύο φορές μέσα στον πίνακα. Τέλος χρησιμοποιούμε QuickSort ώστε να ταξινομήσουμε τις ακμές όσο πιο γρήγορα γίνεται σε χρόνο nlogn.

Χρόνοι και παρατηρήσεις:

- Ο Δοκιμή ενός απλού γράφου για εξέταση βασικών λειτουργιών με λίγους κόμβους και ακμές:



Με βάση τις εντολές:

Τα αποτελέσματα είναι τα εξής:

```
BUILD GRAPH a.txt
COMPUTESPANNINGTREE GRAPH
COMPUTESHORTESTPATH GRAPH 1 7
GETSIZE GRAPH
FINDCONNECTEDCOMPONENTS GRAPH
INSERT GRAPH 7 8 2
COMPUTESHORTESTPATH GRAPH 1 8
DELETE GRAPH 7 8
GETSIZE GRAPH
```

```
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ BUILD: 0.000164
Minimum spanning tree is : 16
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ COMPUTESPANNINGTREE: 0.000102
Shortest path is : 6
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ COMPUTESHORTESTPATH: 1e-06
Size of graph is : 7 12
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ GETSIZE: 1e-06
Number of connected components are: 1
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ FINDCONNECTEDCOMPONENTS: 2e-06
7 8 2
Ο κόμβος προστέθηκε με επιτυχία
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ INSERT: 7.7e-05
Shortest path is : 8
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ COMPUTESHORTESTPATH: 1e-06
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ DELETE: 0.000119
Size of graph is : 7 12
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ GETSIZE: 1e-06
```

Παρατηρούμε όντως ότι το συνολικό βάρος του MST είναι 16

καθώς και το μέγεθος του γράφου και η απόσταση του 1 από το 7 είναι σωστές . Επίσης ο γράφος έχει όντως μια συνιστώσα. Τέλος, με τη προσθήκη του κόμβου 8 η απόσταση του κόμβου 1 από τον 8 θα είναι η απόσταση 1-7 + την ακμή 7-8 όπου και όντως 6 + 2 μας κάνει το αποτέλεσμα που βλέπουμε στο αρχείο =8